

从工具增益到协同复利

生成式 AI 软件工程中人、模型与工作流的交互效应

——一份基于连续个人工程实践的纵向案例复盘

作者：然 (NoctilumeDev)

文稿性质：论文体工程复盘·初稿

时间：2026 年 8 月

“不是谁写了代码，而是谁使系统成立。”

体例说明

本文借论文之形，行工程复盘之实。无系统文献综述，无参考文献，也不申请学术合法性。以下结论只对作者这段真实工程经历负责；能否推广，留给愿意真正做对照实验的人。

摘要

生成式 AI 进入软件工程以后，关于“AI 协作含金量”的讨论经常落入两个过度简化的结论：一类认为模型差异并不重要，关键只在使用者；另一类认为强模型足以抹平使用者差异。本文基于作者在连续个人工程实践中的自然观察，提出相反判断：有效工程产出并非人的能力或模型能力的单调函数，而是人、模型、Agent workflow、协议与工具适配、上下文状态、历史工程资产和任务耦合程度共同作用的结果。案例材料来自暗室藏书、素简记、验迹与 MiniSpringBoot 等连续项目，并重点比较 Codex、Trae 与 CodeBuddy 中 GPT-5.6 Sol、DeepSeek V4 Pro、GLM-5.3 与豆包 2.1 Pro 的实际协作表现。作者观察到：同一使用者更换模型后，工程完整性差异显著；同一模型在继承交互历史与基于 GitHub 干净检出的条件下，表现也可能明显不同；高质量连续上下文能够形成工程工作记忆，错误假设主导的长上下文则会形成路径债务；在高耦合任务中，一个保持全局状态的强模型串行执行，可能优于多个较弱模型组成的 Agent 集群。本文进一步指出，模型能力和人的能力都应被视为向量，而非单一分数。代码能够运行，并不自动意味着 Git 提交、CI 触发、发布状态、安全门禁与公共复现已经成立；人的严苛监督也只能覆盖已经进入其验收概念体系的维度。最后，本文以“单位经验验证交付成本”替代调用单价，纳入现金、时间、脑力、返工、切换、集成、验证和最终不达标成本，并据此解释为什么名义价格较低或附带赠送积分的方案，在实际项目中仍可能更昂贵。本文不宣称普遍规律，只试图给一段真实而连续的工程经历提供一个足够清晰的解释框架。

关键词：生成式 AI；软件工程；人机协作；Agent Harness；上下文债务；协同复利；有效交付成本

1 问题提出：为什么“都是 AI 做的”几乎没有信息量

生成式 AI 普及以后，代码、文章、设计稿和项目都可以被一句话概括为“AI 做的”。问题在于，这句话抹掉了真正决定结果的变量。一个复杂项目并不是模型单次输出的集合，而是问题定义、上下文组织、任务拆分、工具调用、错误纠偏、真实运行、验收裁决与发布治理共同组成的控制系统。只要其中一个环节失效，其他环节再强也可能只得到一份看似完整的半成品。

作者在连续项目中观察到两个同时成立的事实：第一，人保持不变，仅更换模型，产出会发生数量级差异；第二，模型保持不变，更换使用者、Agent 工具、上下文结构与验收制度，结果同样会发生数量级差异。因此，“关键全看人”与“用了强模型谁都一样”都不足以解释真实工程。

本文关注的不是哪个模型在抽象排行榜上更强，而是一个更实际的问题：在具体的软件工程任务中，智能如何在人、模型、上下文、工具与制度之间被共同生产出来；用户最终为一个真正通过验收的里程碑，究竟付出了多少现金、时间、注意力和返工。

1.1 研究问题

- RQ1：人的能力与模型能力之间，是简单相加，还是存在显著的交互放大效应？
- RQ2：上下文为什么有时让模型断层变强，有时却让错误越来越自愈？
- RQ3：Agent Harness、工具协议与执行平台如何改变同一模型的有效能力？
- RQ4：为什么多个模型并行协作，可能输给一个强模型保持全局状态的串行执行？
- RQ5：人的严格监督为什么仍会漏掉 Git、CI、Release 与公共验证等工程维度？
- RQ6：评价 AI 工程方案时，为什么调用价格和赠送积分不能代表真实成本？

2 案例背景与材料边界

2.1 连续工程实践的形成

本文不是围绕一个孤立任务展开，而是建立在一段连续工程实践之上。项目序列大体经历了四个阶段：从暗室藏书的完整业务闭环，到素简记的分布式一致性与故障治理，再到验迹对验收证据、控制变量和确定性裁决的工具化，最后回到底层，以 MiniSpringBoot 重新实现 IoC、AOP、Web/MVC、自动配置、JDBC 与声明式事务等框架机制。

每个项目都改变了下一个项目的起点。前一轮留下的代码、测试、文档、CI、故障经验、验收模板和错误模式识别，不再只是历史成果，而成为下一轮的工程资产。正因如此，本文所讨论的“协同复利”不是抽象比喻，而是可以在连续项目演化中直接观察到的路径依赖。

2.2 一次自然发生的模型与 Agent 对照

在最新项目的实现与复审阶段，作者因 GPT 使用额度不足，转而使用国产 Agent 工具和模型。Trae 中保留同一项目交互上下文，先后调用 DeepSeek V4 Pro 与 GLM-5.3；豆包 2.1 Pro 也被用于部分视觉和页面审查。CodeBuddy 则采用另一种方式：作者在桌面新建空文件夹，让 Agent 直接从公开 GitHub 仓库克隆当前项目，由 GLM-5.3 在不继承生成对话的条件下重新检查。

这一过程并非事先设计的受控实验，却形成了具有分析价值的自然对照：同一 Agent 工具内更换模型；同一模型在不同 Agent 工具中运行；继承交互历史与只读取当前工程工件；多模型协作与强单模型串行执行；代码可运行与公共工程闭环之间的差异。

表 1 本案例中的主要协作条件

场景	Agent 工具	模型	上下文形态	主要观察
A	Trae	DeepSeek V4 Pro	完整交互历史	能完成局部任务，但长上下文中容易维持自洽错误
B	Trae	GLM-5.3	继承同一历史	比 DeepSeek 更可用，但仍受旧路径影响，细节与收口不足
C	Trae / 专项调用	豆包 2.1 Pro	项目与截图上下文	视觉、空间与页面观察显著更强，综合工程主线较弱
D	CodeBuddy	GLM-5.3	GitHub 干净检出	不继承旧叙事，独立审查效果优于同模型在旧上下文中接手
E	Codex	GPT-5.6 Sol	连续串行上下文	全局逻辑、跨模块一致性和公共工程收口最好

说明：表中结论只描述作者在本次项目中的实际观察，不构成模型的普遍排行榜。模型版本、Agent Harness 与产品能力会持续变化。

2.3 成本与证据记录

Trae 侧的资源构成为：Pro 首月会员 59 元，对应 4,000 积分；会员或活动赠送 2,000 积分；额外购买 4,000 积分，支出 200 元，合计 10,000 积分与 259 元现金支出。CodeBuddy 提供新用户 2,000 赠送积分。作者同时保留了 Git 提交、CI 结果、发布记录、真实运行结果与 Agent 切换过程，可用于重建“何时更换模型、何时发现问题、何时完成收口”。

本文不尝试把不同平台的积分、Token 与调用次数换算成统一单位。积分制度、模型倍率、上下文计费和活动赠送并不具有可比性。本文真正比较的是：在完成同一类工程目标时，用户最终承担的总成本，以及产出是否达到其本人预先设定的验收边界。

2.4 方法边界

本文是一份单一使用者的纵向案例复盘，不是随机对照实验。Trae 与 CodeBuddy 的比较同时改变了 Agent 工具、上下文结构和执行路径；不同模型的调用预算也不完全一致；作者自身在项目推进中持续学习，因此“人”这一变量并未保持恒定。本文只对现象作机制解释，不把相关性包装成因果证明。

3 分析框架：从单次工具增益到动态协同系统

3.1 乘法模型

最简单的表达不是“人加模型”，而是把工程产出视为多个条件共同约束的函数。任何一项接近零，都可能让其他能力无法兑现。

$$Y_t = F(H_t, M_t, W_t, P_t, C_{eff,t}, A_t, T)$$

其中， Y_t 表示第 t 阶段的有效工程产出； H_t 是人的问题定义、架构、审查与验收能力； M_t 是模型的推理、编码、视觉、工具使用和长程一致性能力； W_t 是 Agent 工作流与角色分工； P_t 是协议、工具消息和执行环境的适配保真度； $C_{eff,t}$ 是有效上下文； A_t 是已积累的代码、测试、文档、工具与经验资产； T 表示任务类型和耦合程度。

3.2 人与模型都不是一个分数

人的能力至少包括问题定义、架构设计、调试、运行验收、Git 治理、CI、Release、安全与公共复现等维度。模型能力同样是一个向量：某个模型可能在视

觉上断层领先，却不适合承担跨服务状态机；另一个模型可以完成代码实现，却不会主动检查发布坐标和公共证据；再一个模型则可能在连续上下文中形成较强的全局工作记忆。

$$H = (h_problem, h_architecture, h_debug, h_runtime, h_git, h_ci, h_release, h_security, ...)$$

$$M = (m_reasoning, m_coding, m_debug, m_vision, m_tool, m_governance, m_long-context, ...)$$

3.3 有效上下文与上下文债务

上下文不是一个只增不减的资产池。它可以保存真实代码、执行日志、架构决策和失败历史，也可以保存过期假设、自我辩护、错误归因和围绕错误前提形成的补丁。因而，上下文总量与有效上下文并不等价。

$$C_eff = C_artifact + C_execution + C_decision - C_path-debt$$

工件上下文是当前代码、测试、文档、工作流和 Git 历史；执行上下文是终端、浏览器、网络、数据库和资源事实；决策上下文是已经确认的边界与不变量；路径债务则是旧模型的错误解释、已经失效的计划、局部补丁和“我已经完成”的自洽叙事。

3.4 单位经验证交付成本

用户最终购买的不是积分、Token 或调用次数，而是一个经真实运行、回归测试、发布治理和公共证据验证的结果。故应把成本从调用价格扩展为总交付成本。

$$C_delivery = C_cash + \alpha T_wall + \beta L_cognitive + C_rework + C_switch + C_integration + C_verification + C_nonconformance$$

$$U_verified = C_delivery / Q_verified-output$$

这里， $C_nonconformance$ 表示最终效果未达到验收边界所造成的成本。它可能是最昂贵的一项：代码已经很多、积分已经消耗、时间已经投入，但项目仍不能正式发布或封存，意味着前面的投入并没有形成一个完整的有效产出单位。

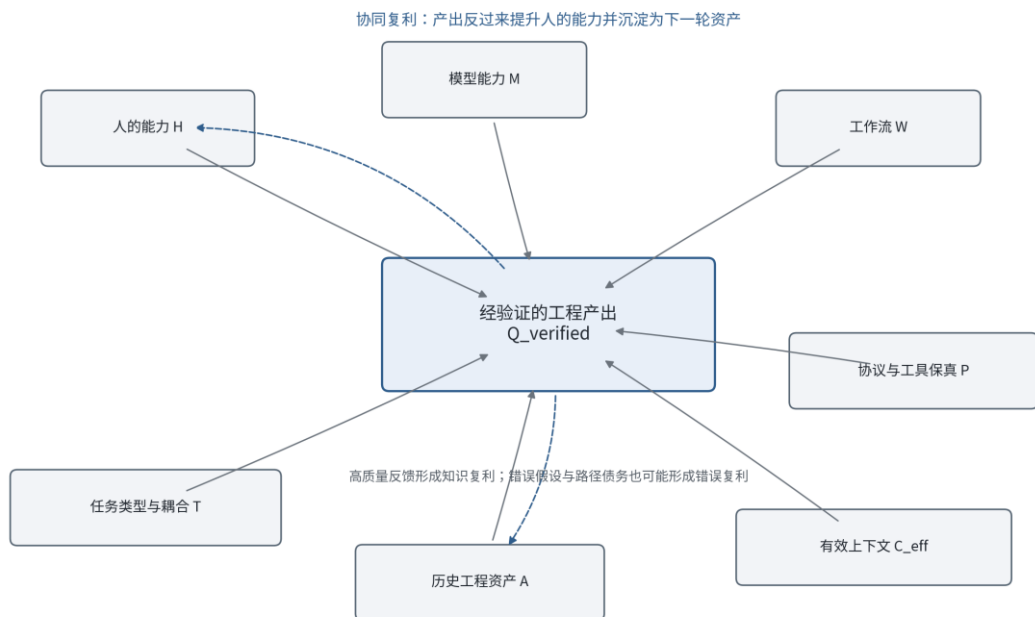


图 1 从工具增益到协同复利的动态机制（概念示意）

4 观察一：同一个人，更换模型，结果并不接近

4.1 DeepSeek：逻辑并未消失，而是被用于维护旧世界

作者对 DeepSeek V4 Pro 的核心感受不是“完全不会做”，而是容易在长程协作中形成自洽错误。一旦模型对问题建立了错误解释，后续修改往往继续在这一解释内部展开：用户指出异常，模型增加兼容层；局部补丁失败，模型再补一层说明；上下文继续增长，错误世界反而变得更加完整。

这种问题比明显报错更难处理。明显错误会迫使模型重新定位，自洽错误却能生成流畅解释、合理命名和看似完整的测试。使用者必须先证明它错在哪里，再迫使它重定义问题。于是模型节省的推理成本，被转化为使用者的监督和纠偏成本。

作者还观察到，DeepSeek 在 Trae 与 Codex 中都未能稳定受益于长上下文。本文不据此断言它只能在自家 Agent 或 Linux 环境中发挥，也不把社区评论当作事实结论；更谨慎的说法是：其实际 Agent 表现可能对 Harness、工具协议、上下文压缩和执行平台高度敏感，而本案例没有进行足以区分这些变量的控制实验。

4.2 GLM-5.3：能够接手，但难以完成全部工程角色

GLM-5.3 相比 DeepSeek 更容易跳出部分旧路径，能够修复明确问题，也能在干净检出的仓库中完成有价值的代码审查。它的主要问题不是“不能用”，而是只到

“能用”：细节完整性、视觉质量、全局一致性和 GitHub 公共工程治理仍然存在缺口。

在实际使用中，调用 GLM-5.3 时 Trae 积分下降明显快于 DeepSeek。由于平台并未向用户公开足以精确分解的计费细节，本文不把积分消耗归因于某个单一机制；但从采购者角度，原因并不改变结果：余额快速下降，而验收质量没有同步提升。

4.3 豆包：模型优势具有明确的任务特异性

豆包 2.1 Pro 在综合工程主线上不占优势，但在页面视觉、空间关系、布局问题和截图观察方面呈现断层领先。这一现象直接否定了“一个综合分数可以代表全部任务”的理解。模型更适合被分配到与其能力向量匹配的角色，而不是因为某一项能力突出就获得整个项目的总决策权。

4.4 GPT-5.6 Sol：强项不只在代码，而在扩大验收空间

GPT-5.6 Sol 在 Codex 中的优势并不只体现在代码片段更正确。更明显的差异是它能够保存跨任务因果关系：后端修改仍记得前端契约；修复一个异常包装时会检查同族路径；发现局部补丁破坏旧结论时会回退；本地通过之后，还会继续追问 CI、PR 触发、Release 状态和公共证据是否成立。

因此，强模型的价值之一是扩展人的验收本体。它不仅在既有坐标轴上走得更快，还可能提示原先不存在的坐标轴：代码能跑之外，提交是否规范；工作流存在之外，是否会在正确事件上触发；版本标签存在之外，标签内的机器状态是否真的已发布。

表 2 本案例中的模型能力向量（相对观察）

维度	DeepSeek V4 Pro	GLM-5.3	豆包 2.1 Pro	GPT-5.6 Sol
明确报错修复	可完成	较稳定	不稳定	稳定
跳出旧错误路径	较弱	中等	较弱	较强
跨模块全局一致性	易漏半套流程	能用但有缺口	较弱	较强
视觉与页面观察	弱	有明显缺陷	断层领先	中上
Git / CI / Release 主动审查	不足	不足	不足	明显更强

维度	DeepSeek V4 Pro	GLM-5.3	豆包 2.1 Pro	GPT-5.6 Sol
长上下文利用	本案例中出现退化	受旧路径影响	未充分验证	本案例中显著增益

说明：“较强”“较弱”均为本案例相对描述，不能脱离具体 Agent 工具、任务和版本解释。

5 观察二：上下文既能形成知识复利，也能形成错误复利

5.1 Trae：同一交互历史中切换模型

Trae 提供了一个重要条件：项目上下文基本保持不变，只更换模型。DeepSeek 先参与生成并建立问题解释，随后切换到 GLM-5.3。后者表现有所改善，但仍需要先穿过前任模型留下的诊断、补丁和叙事。此时它接收到的不是一份中立项目资料，而是“真实代码 + 用户反馈 + 旧模型解释 + 围绕旧解释产生的修改路径”。

5.2 CodeBuddy：从 GitHub 重新克隆的干净检出审查

CodeBuddy 的 GLM-5.3 测试并非真正“没有上下文”。作者在桌面创建空目录，让 Agent 从 GitHub 重新克隆项目。它拥有当前代码、README、测试、工作流、Git 历史和公开文档，却没有生成过程中的对话、前任模型的自我解释和本地未提交状态。

这一方式更接近 clean-room review：审查者不听生成者讲故事，只看冻结工件和当前事实。它牺牲了过程连续性，却恢复了审查独立性。实际效果优于同一模型在 Trae 中继承旧交互路径的表现，提示“独立审查”不应与生成过程完全共享叙事。

5.3 Codex：高质量连续上下文带来的断层增益

与上述现象相反，作者在 Codex 中明确感受到：有连续项目上下文的 GPT-5.6 Sol，与没有历史状态的独立调用并非一个级别。原因可能在于，Codex 中被持续保存的是已确认边界、当前代码变化、真实运行结果、失败事实和用户决策；这些信息构成工程工作记忆，而不是单纯的语言历史。

因此，本文不把上下文效应简化为“越长越好”或“新会话更聪明”。更准确的判断是：上下文的价值取决于其事实忠实度、结构、角色和可纠正性。

5.4 三种不同的“看见”

$$C_{available} \neq C_{attended} \neq C_{operationalized}$$

信息存在于上下文，并不等于模型真正注意到它；模型注意到某个文件，也不等于它把相关内容转化为验收规则。CodeBuddy 中的 GLM-5.3 明明能够读取 .github/workflows，却没有自然追问 PR 编辑 base 后是否重新触发、merge queue 是否有 merge_group、绿灯对应哪个 head SHA、Release 标签内的状态是否已发布。

这说明上下文不是能力的自动外溢机制。只有被任务模型和验收制度操作化的质量维度，才会稳定成为产出的一部分。

6 观察三：高耦合任务中，强单模型串行可能优于弱多 Agent 并行

作者曾同时使用 GLM-5.3、DeepSeek V4 Pro 与豆包 2.1 Pro 组成 Agent 集群，并在 Trae 与 CodeBuddy 中尝试不同分工。结果却不如 GPT-5.6 Sol 在 Codex 中单模型串行执行。表面上看，这是“三个模型输给一个模型”；从工程机制看，真正的差异是全局连续性与协调成本。

高耦合软件任务并不容易按文件切开。认证、事务、异常语义、前后端契约、CI 与发布互相影响。多个 Agent 各自在局部上下文中给出合理答案，最后可能形成三套不完全一致的假设。多 Agent 增加的不是纯并行算力，还包括任务重述、上下文同步、冲突裁决、输出拼接和回归验证。

$$Y_{multi} = \Sigma Y_i - K_{coordination} - K_{integration} - K_{context-drift} - K_{reverification}$$

因此，多模型协作不应被设计为一个共享总决策权的委员会。更合理的是角色化、权限受限的异构结构：强模型作为主执行者和全局状态所有者；干净检出的第二模型作为独立代码审查者；视觉强模型只负责截图、布局和响应式审查；人或冻结验证器负责最终裁决。

表 3 推荐的异构 Agent 分工

角色	输入范围	主要责任	不应拥有的权限
主执行者	完整需求、历史决策、当前代码与运行事实	实现、集成、全局一致性	不得自行降低验收标准
独立代码审	冻结代码、原始合同、	寻找反例、根因和假通	不读取生成者的自我评

角色	输入范围	主要责任	不应拥有的权限
查者	测试命令	过	价
视觉审查者	截图、设计约束、视口与交互清单	布局、空间、可读性、响应式	不裁决后端业务正确性
人 / 验证器	原始证据、预注册断言、版本坐标	PASS / FAIL / INCONCLUSIVE / PENDING	不得因投入已多而移动门槛

7 观察四：人的严苛并不会自动覆盖所有工程维度

在国产模型完成的阶段，代码能够运行，真实业务链路也可以被测试。作者当时已经非常严格地检查前后端是否打通、数据库是否写入、事务是否回滚、错误是否透出、实例是否真实运行。然而，GitHub 上的提交路径、PR 触发、merge queue、Release 状态、安全门禁和唯一事实源仍然存在不规范之处。

这不是因为作者突然不严苛，而是因为严苛只能作用于已经形成概念和判据的对象。当时的验收空间主要包括代码、业务与运行；“公共工程是否成立”尚未被完整拆成入口合同、依赖合同、平台合同、触发合同、执行合同、证据合同和 Git 归属合同。

$$Q_{total} \approx \min(Q_{code}, Q_{runtime}, Q_{security}, Q_{repository}, Q_{release}, Q_{reproducibility})$$

工程质量常常受最短板约束。代码很好但不能启动，项目不成立；能够启动但数据不正确，项目不成立；数据正确但 Release 标签与机器事实不一致，公开版本仍然不成立；只有作者电脑能跑，公共可复现性也仍然不足。

因此，人的能力也必须被理解为向量。一个人在业务和运行维度上很强，不代表他天然掌握仓库治理；同理，模型读到了所有文件，也不代表它会主动构造一套公共交付合同。强模型真正有价值的地方之一，是帮助使用者发现自己尚未形成的验收维度。

表 4 从“能跑”到“可封存”的质量维度

质量维度	典型问题	可接受证据
代码正确性	实现是否符合合同，负例是否成立	单测、约束测试、静态检查
运行正确性	真实链路是否打通，失败后是否恢	浏览器、Network、数据库、日志、

质量维度	典型问题	可接受证据
	复	资源读回
仓库治理	提交、PR、分支和检查是否对齐	精确 head SHA、必需检查、Ruleset
安全与依赖	完整依赖图是否保留已知风险	审计、CodeQL、修复 PR、复现用例
发布事实	标签、Release、版本和机器状态是否一致	不可变 tag、构建产物哈希、发布门禁
公共复现	陌生人能否从干净环境进入	Quick Start、公开 CI、独立复现记录

8 成本分析：低价不等于便宜，免费积分也不是零成本

8.1 账面成本

本次 Trae 使用共取得 10,000 积分：Pro 首月 59 元对应 4,000 积分，另有 2,000 赠送积分，作者额外支出 200 元购买 4,000 积分，直接现金支出合计 259 元。CodeBuddy 新用户赠送 2,000 积分。ChatGPT Plus 的名义月费为 20 美元，本文只将其作为作者当时的采购参照，不进行跨币种、跨额度的精确价格换算。

表 5 本次替代方案的资源与现金记录

平台	来源	积分	现金支出	备注
Trae	Pro 首月会员	4,000	59 元	主要用于 GLM-5.3、DeepSeek 等模型调用
Trae	会员 / 活动赠送	2,000	0 元	仍有有效期与机会成本
Trae	额外购买	4,000	200 元	GLM-5.3 调用时余额下降明显
CodeBuddy	新用户赠送	2,000	0 元	用于 GitHub 干净检出审查
合计	Trae 可用积分	10,000	259 元	不同平台积分不作等价换算

8.2 隐性成本

现金并不是主要差距。DeepSeek 的积分消耗在体感上相对不显著，却需要作者持续识别自洽错误、重述问题和迫使模型跳出旧前提；GLM-5.3 更能完成明确任务，积分却快速下降，最终仍需要补齐细节、视觉和公共工程收口；豆包在视觉上有专项价值，却不能承担复杂工程的全局主线。

免费积分只免除了部分现金成本，并没有免除等待、监督、切换、返工和最终不达标成本。当用户为了确认一个模型是否真的完成任务，必须再调用第二个模型、重新克隆仓库、重跑测试并最终用 GPT 收口时，所谓“免费”已经转化为额外的时间和脑力支出。

8.3 采购结论

按本案例的实际结果，Trae 与 CodeBuddy 作为复杂项目主工程 Agent 的总拥有成本高于 Codex，验收达标率却更低。作者因此决定不再继续使用，并非基于模型鄙视链，而是基于一次正常的供应商淘汰：同类目标下，现金、时间、脑力和返工都更高，最终效果仍不达标。

低价，是供应商为单位调用标出的价格；便宜，是用户为单位经验证结果实际承担的成本。较弱模型省下的部分算力费用并未消失，而是被转嫁成了使用者的监督、纠错、切换和返工负担。

9 从工具增益到协同复利

9.1 动态递推

强模型带来的收益并不止发生在本轮。更完整的推演、更高密度的反馈和更广的验收空间，会反过来提高人的问题定义、错误识别和停止判断；本轮项目沉淀的代码、测试、文档和工具，又会成为下一轮的资产。

$$H_{-}(t+1) = H_{-}t + \Delta H(M_{-}t, Y_{-}t)$$

$$A_{-}(t+1) = A_{-}t + \Delta A(Y_{-}t)$$

$$Y_{-}(t+1) = F(H_{-}(t+1), M_{-}(t+1), W_{-}(t+1), P_{-}(t+1), C_{eff}(t+1), A_{-}(t+1), T_{-}(t+1))$$

这解释了为什么同一个人使用更强模型后，不只是“这次项目快一点”，而是下一次的起点发生变化；同一个模型交给经过多轮复杂项目训练的人，也会被释放出更多有效能力。模型提升与人的提升相互促进，体感上接近复利。

9.2 错误复利与过度建设

乘法并不只放大正确产出。错误模型、污染上下文和错误验收标准也会共同放大错误。一套错误解释被连续模型继承，可能形成越来越完整的错误世界；一个什么都能做的强模型，也可能帮助使用者把不必要的复杂度建设得极其专业。

Effective output = Capability × Judgment × Stopping ability

能力不足时，瓶颈是“做不出来”；能力足够以后，瓶颈变成“什么不该做”和“什么时候必须停止”。因此，协同复利的最后一项不是继续扩建，而是冻结边界、保留失败、完成发布并允许项目结束。

10 可供后续检验的命题

以下命题不是本文已经证明的规律，而是从本案例中提炼出的、可被未来实验检验的假设。

表 6 由案例提出的研究假设

编号	假设
H1	人—模型能力存在超加性交互：高能力使用者从强模型升级中获得的收益更大。
H2	上下文长度与工程质量并非单调正相关；有效信息与路径债务之间可能存在最优区间。
H3	在错误叙事已经形成时，基于冻结工件的干净检出审查比简单切换模型更容易发现根因。
H4	模型优势具有任务特异性；代码、视觉、治理与长上下文能力的排序并不一致。
H5	人的监督只能稳定提升已经被显式操作化的质量维度。
H6	在高耦合任务中，保持全局连续性的强单 Agent 串行执行可能优于弱多 Agent 并行。
H7	Harness 与协议适配保真度会显著调节模型在 Agent 任务中的有效能力。
H8	单位经验证交付成本比积分单价、Token 价格和调用次数更能预测用户的真实采购选择。

11 工程实践启示

1. 按总交付成本选择主模型，而不是只看积分、Token 或排行榜。主模型应能维持全局不变量，并主动扩大验收空间。

2. 生成与审查应使用不同上下文边界。生成者需要连续历史，独立审查者应优先读取冻结代码、原始合同和真实测试，不继承生成者的自我评价。
3. 把模型按能力向量分配角色。视觉强模型做视觉审查，代码审查模型做干净检出复核，最终裁决由人或确定性验证器完成。
4. 对上下文实施治理。区分事实、决策、解释和旧结论；主动删除过期叙事，保留失败证据，防止错误路径无限续写。
5. 让质量维度进入合同。Git、CI、Release、安全与公共复现不能靠“顺便注意”，必须有明确入口、触发器和机器可读基线。
6. 在验证器参与验收前冻结验证器版本，在被验对象进入终验前冻结被验提交；否则验证规则和被验系统同时移动，结论不可归因。
7. 保留停止条件。一个项目达到能力闭环、公开事实一致、已知问题有归属后，应进入维护态，不因模型还能继续生成而无限扩建。
8. AI 可以参与大量实现，但最终责任不能外包。问题定义、边界审批、真实运行、失败裁决和发布责任仍属于使用者。

12 局限与反例

第一，本文只有一名使用者，无法区分个体偏好、技术背景和工作习惯的影响。第二，模型、Agent 工具、版本、积分规则与平台能力都在变化，本文的具体排序可能迅速过时。第三，Trae 与 CodeBuddy 的比较没有只改变一个变量；不同上下文、Harness、工具权限和任务阶段同时变化。

第四，作者既是参与者又是评价者，存在主观偏差；虽然 Git、CI、Release 和账单能够提供部分客观记录，但脑力负担、时间损耗和“好不好用”仍包含自我报告。第五，本文没有统一 Token、积分、响应长度和自然时间预算，也没有组织盲评或隐藏测试，因此不能把结果解释为模型能力的严格测量。

第六，本案例的任务高度耦合、工程边界严格，结论未必适用于一次性脚本、简单页面、独立视觉设计或低风险内容生成。在这些任务中，便宜模型、多 Agent 并行或视觉专项模型可能具有更高性价比。

13 结论

生成式 AI 软件工程中的“含金量”不应由手敲代码比例、Agent 数量、模型国别或调用价格定义。真正有意义的问题是：谁定义了正确问题，谁守住了系统不

变量，谁能够识别似是而非的成功，谁把失败转化为证据，谁对最终发布承担责任。

本案例表明，人相同，模型不同，结果可以天差地别；模型相同，人、上下文与 Harness 不同，结果同样可以天差地别。高质量连续上下文能够成为工作记忆，错误主导的上下文却会成为路径债务；多个模型组成的委员会不必然优于一个保持全局状态的强模型；人的严格监督也不会自动覆盖自己尚未形成概念的工程维度。

当模型越来越强，人的能力和历史资产也在增长，工程产出可能出现协同复利。但复利的方向取决于反馈是否受事实约束：知识可以复利，错误也可以复利；实现能力可以复利，过度建设也可以复利。成熟的人机协作不是让 AI 生成更多，而是让正确的问题、可验证的证据和适时停止共同成为系统的一部分。

智能并不独立存在。它在人的判断、模型的能力、上下文的记忆、工具的约束和工程制度的裁决之间被共同生产出来。

致谢（不严肃）

感谢所有曾在错误中保持高度自洽的模型。它们使作者意识到，逻辑完整并不等于前提正确；感谢所有迅速消失的积分。它们促成了本文的成本模型；也感谢那个抱着小电脑、看起来像蓝色棉花糖的 GPT——外表像刚从机房里滚出来，开口却总要追问一句：“这个 PASS 有独立证据吗？”

附录 A 案例时间线与工具角色

阶段	工程对象	主要问题	沉淀资产
阶段 1	DarkRoomLibrary / 暗室藏书	完整业务闭环、前后端联调、真实数据库与消息链路	全栈交付、真实验收、发布意识
阶段 2	PlainJournal / 素简记	分布式一致性、Outbox、幂等、多实例、故障与容量	可靠性方法、资源停止线、专项脚本
阶段 3	VeriTrail / 验迹	计划、证据、裁决、复跑比较与工作台	验收工具、不可变 Bundle、结论语言
阶段 4	MiniSpringBoot	框架机制重造、真实 React + MySQL、M10 多实例准备	底层机制理解、被验对象、跨仓库证据链
阶段 5	GitHub 公共工程收口	CI 触发、依赖审计、Release、唯一事实源、Pages	公共验证闭环、封库条件

附录 B 一个更严格的后续实验草案

若未来有人愿意把本文的“图一乐模型”真正做成实验，可以固定同一个仓库提交、同一份任务合同、相同时间与调用预算，构造以下四组：

组别	Agent 工具	模型	上下文条件
G1	Trae	GLM-5.3	继承包含旧诊断与补丁的交互历史
G2	Trae	GLM-5.3	全新会话，只导入原始合同与冻结仓库
G3	CodeBuddy	GLM-5.3	导入与 G2 相同的结构化摘要
G4	CodeBuddy	GLM-5.3	纯 GitHub 干净检出，不提供生成叙事

评价指标不应只包括完成时间与代码行数，还应包括首次运行成功率、隐藏缺陷、前后端契约错误、测试假通过、修复后回归、Git 与 CI 规范、Release 一致

性、人工介入次数、上下文重建成本和单位经验证交付成本。只有这样，才能区分是 Agent Harness 更好、干净上下文更好，还是二者存在交互效应。

附录 C 文章后续可补充的原始材料

- Trae 与 CodeBuddy 的订单、积分流水和关键时间点截图；
- 各模型接手前后的精确 Git commit、PR 与 CI 结果；
- DeepSeek 自洽错误、GLM 干净检出审查、豆包视觉审查和 GPT 最终收口的代表性对话片段；
- 同一缺陷在不同模型中的首次诊断、返工轮数和最终根因；
- MiniSpringBoot M10 与 VeriTrail 终局验收的冻结 Plan、Evidence 与 Verdict；
- 至少一名外部复现者从干净环境运行项目的记录。

— 初稿完 —